RAJALAKSHMI
ENGINEERING COLLEGE

An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

BONAFIDE CERTIFICATE

NAME ....SHARJIN JINO S A.....

ACADEMIC YEAR ~~2026-2027~~.... SEMESTER ....V.....BRANCH....CSD.....

UNIVERSITY REGISTER No. 2116241701052

Certified that this is the bonafide record of work done by the above student in the

Graphics and multimedia (IT23E31) Laboratory during the year 2026 - 2027

Signature of Faculty - in - Charge

Submitted for the Practical Examination held on .....

Internal Examiner

External Examiner

NAME: Sharjinjino STD.: _____ SEC.: _____ ROLL NO.: _____ SUB.: _____

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	21-7-2026	Bresenham line drawing Algo for $\|m\| < 1$	3	
2.	25-07-2026	Breceuham circle drawing Algoritem	9	
3.	28-07-2026	Bresenham Ellipse drawing Algorithm	13	
4.	04-08-2026	DDA line drawing Algo	15	
5.	11-08-2026	Draw a rectang using Bresenham	17	
6.	11-08-2026	Draw a smiling face using circle	19	
7.	11-08-2026	Draw a olympic logo using ellipse	21	
8.	11-08-2026	Draw a butterfly using ellipse	23	
9.	15-08-2026	Draw a traffic signal using Bresenham	25	
10.	15-08-2026	Flood fill algoritem	27	
12.	15-08-2026	2D transformations	29	
13.	18-08-2026	Polygon clipping	31	
14.	21-08-2026	Basic 3D transformation		

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
15.	5/9/2016	Create and Render 3D objects using polygons and Apply Basic color and shading techniques		
16.	5/9/2016	Create a multimedia messanging App		
17.	8/9/2016	Capture video and Audio from web-cam and Microphone		
18.	8/9/2016	Create Application to recieve or send texts		
19.	22/9/2016	create 3D model using Blender		

completed

Experiment: 1 Bresenham's line drawing Algorithm for $|m| < 1$

Aim:

To implement Brensenham's Line drawing Algorithm for $|m| < 1$ using python programming

PROGRAM:

```
import matplotlib.pyplot as plt

def brensenham_line(x1, y1, x2, y2):
    point = []
    dx = x2 - x1
    dy = y2 - y1
    p = 2*dy - dx
    x = x1
    y = y1
    points.append((x,y))
while x < x2:
    x = x+1
    if p<0:
        p = p+2*dy
    else:
        y = y+1
```

```
        p = p+2 * dy - 2 * dx
        points.append((x, y))
    return points

x1 = int(input("Enter x1: "))
y1 = int(input("Enter y1: "))
x2 = int(input("Enter x2: "))
y2 = int(input("Enter y2: "))

points = bresenham_line(x1, y1, x2, y2)
print("points generated")
    for p in points:
        print(p)

x_values = [p[0] for p in points]
y_values = [p[1] for p in points]

plt.plot(x_values, y_values, marker
    ="o")
plt.title("Bresenham line drawing Algo")
plt.xlabel("x-axis")
plt.ylabel("y-axis")
```

```
plt.grid()
plt.show()
```

21/7/20

Result:

Have implemented Bresenham's Line drawing Algorithm for $|m| < 1$ using pytesor programming.

Enter x1 : 5

Enter y1 : 8

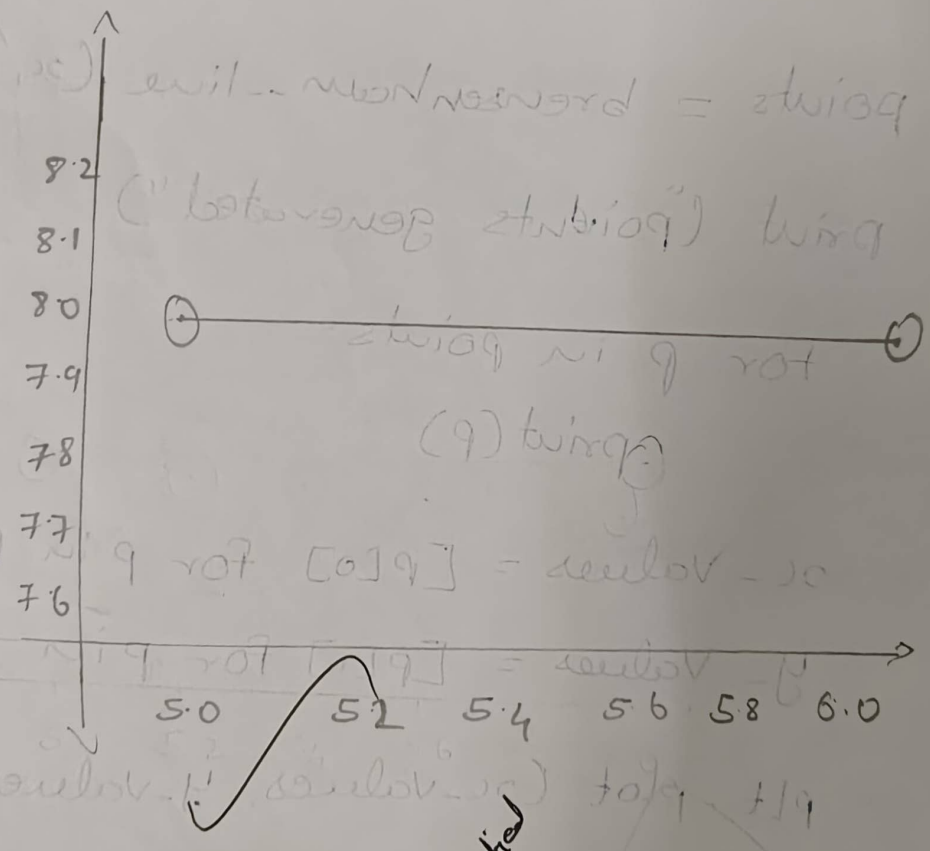
Enter x_2 : 6

Enter y_2 : 2

points generated

(5, 8)

(6, 8)



o/p verified
21/7/24

Exp - 2 Bresenham's circle drawing Algorithm

Aim:

To implement Bresenham circle drawing algorithm using the python programming language

code:

```
import matplotlib.pyplot as plt
def Bresenham_circle(xc, yc, r):
    points = set()
    x = 0
    y = r
    def plot_symmetric_points(x_coord, y_coord):
        (xc+x, yc+y),
        (xc-x, yc+y),
        (xc+x, yc-y),
        (xc-x, yc-y),
        (xc+y, yc+x),
        (xc-y, yc+x),
        (xc+y, yc-x),
        (xc-y, yc-x)
    for p in points
        x.append(p[0])
        y.append(p[1])
def bresenham_circle(xc, yc, r):
```

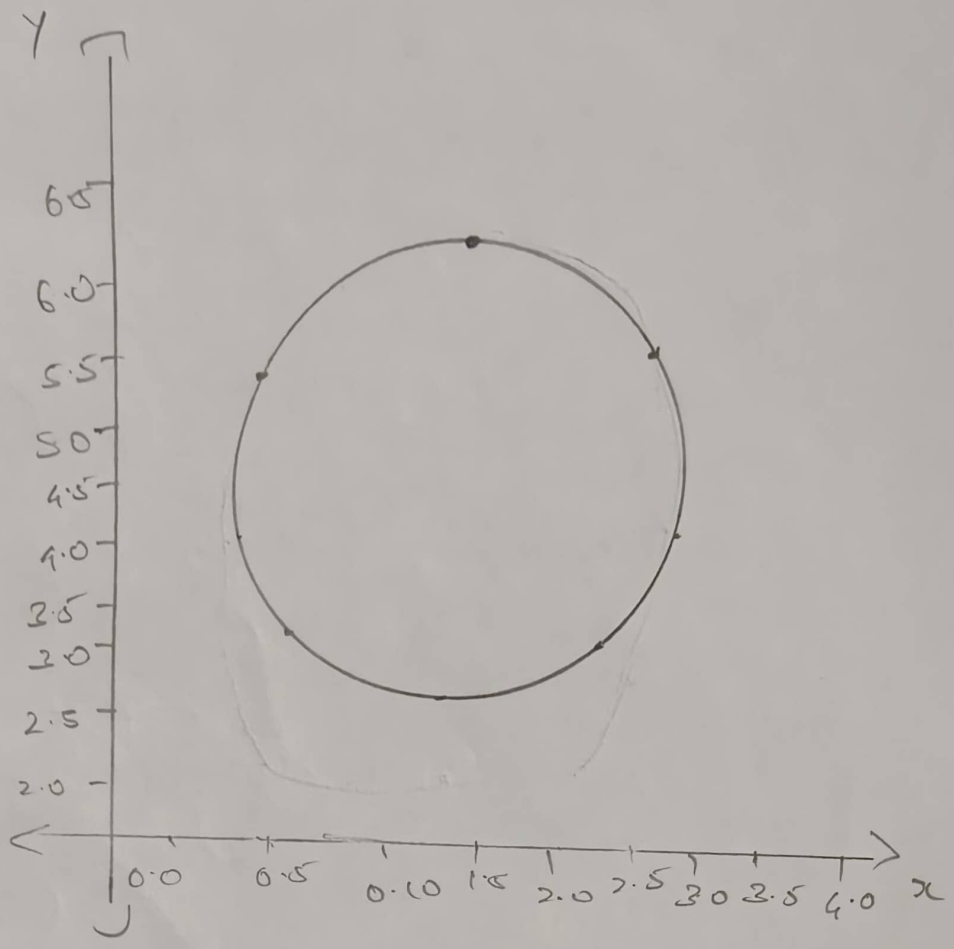
```
x = 0
y = r
p = 3 - 2 * r
xc = []
yc = []
while x <= y:
    plot_points(xc, yc, x, y, z, y)
    if p < 0:
        p = p + 4 * x + 6
    else:
        y -= 1
        p = p + 4 * (x - y) + 10
        x += 1
    return x, y
```

28/7/26

Result:

Thus Bresenham circle drawing algorithm using python programming is executed

OUTPUT:



input :

xc : 2

yc = 2

r = 4

points: (2, 6), (2, 6), (2, -2), (2, -2), (6, 2)

(-2, 2), (6, 2), (-2, 2), (6, 2)

(-2, 2), (3, 6), (1, 6), (3, -2),

(1, -2), (6, 3), (4, 5), (-2, 1)

Exp-3 Bresenham Ellipse drawing Algorithm

Aim:

To implement the Bresenham Ellipse drawing Algorithm in python

Code:

```
import matplotlib.pyplot as py
xc = int(input("center x: "))
yc = int(input("center y: "))
rx = int(input("Radius x: "))
ry = int(input("Radius y: "))

x, y = 0, ry
rx2, ry2 = rx*rx, ry*ry
dx, dy = 0, 2*rx2*y
d1 = ry2 - rx2*ry + 0.25*rx2

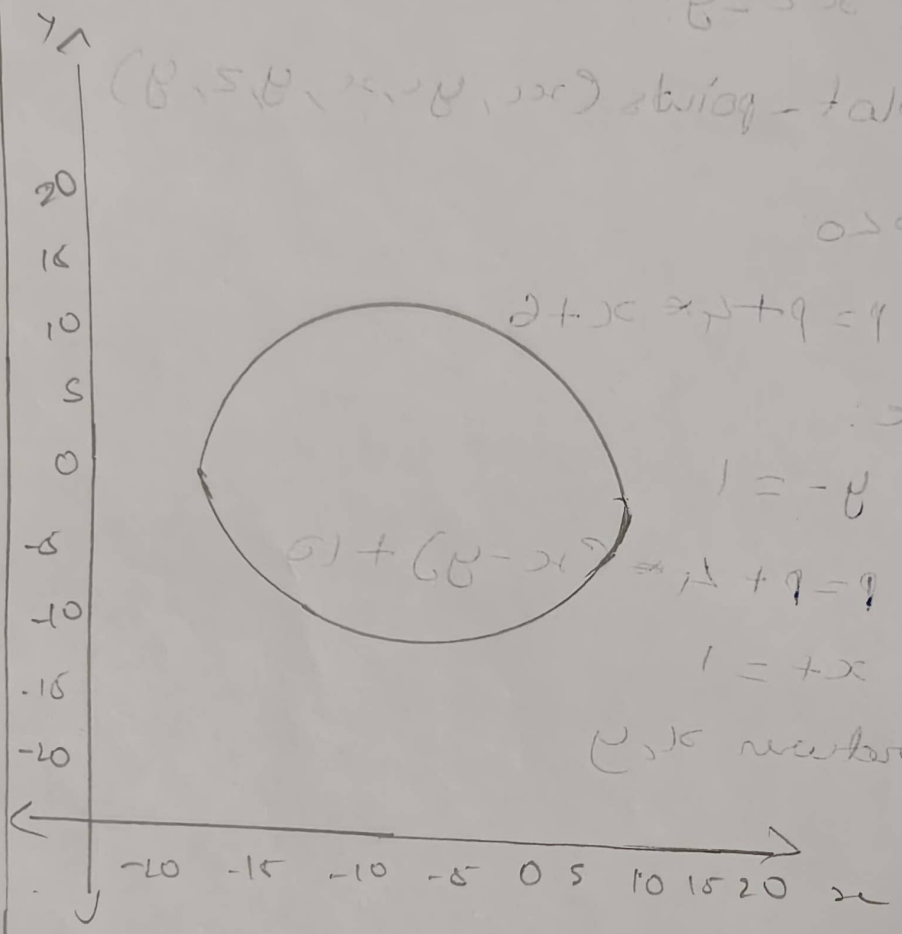
x, y = [], []
def plot(x, y):
    x.extend([x+xc, xc-x, xc+x,
    xc-x])
    y.extend([yc+y, yc+y, yc-y,
    yc-y])
```

```
while dx < dy:
    Plot (x, y)
    if d1 < 0:
        x += 1
        dx += 2 * ry2
        d1 += dx + ry2
    else:
        x += 1
        y -= 1
        dx += 2 * ry2
        dy -= 2 * rx2
        d1 += dx - dy + ry2

d2 = ry2 * (x + 0.5) ** 2 + rx2 * (y - 1) ** 2
     - rx2 * ry2

while y >= 0:
    Plot (x, y)
    if d2 > 0:
        y -= 1
        dy -= 2 * rx2
        d2 += rx2 - dy
    else:
        y -= 1
        x += 1
        dx += 2 * ry2
        dy -= 2 * rx2
        d2 += dx - dy + rx2
```

OUTPUT:



Enter center x: 0

Enter center y: 0

Enter x radius: 20

Enter y radius: 10

Exp 4: DDA Algorithm for implementing line drawing

Aim: To implement line drawing using DDA Algorithms

code:

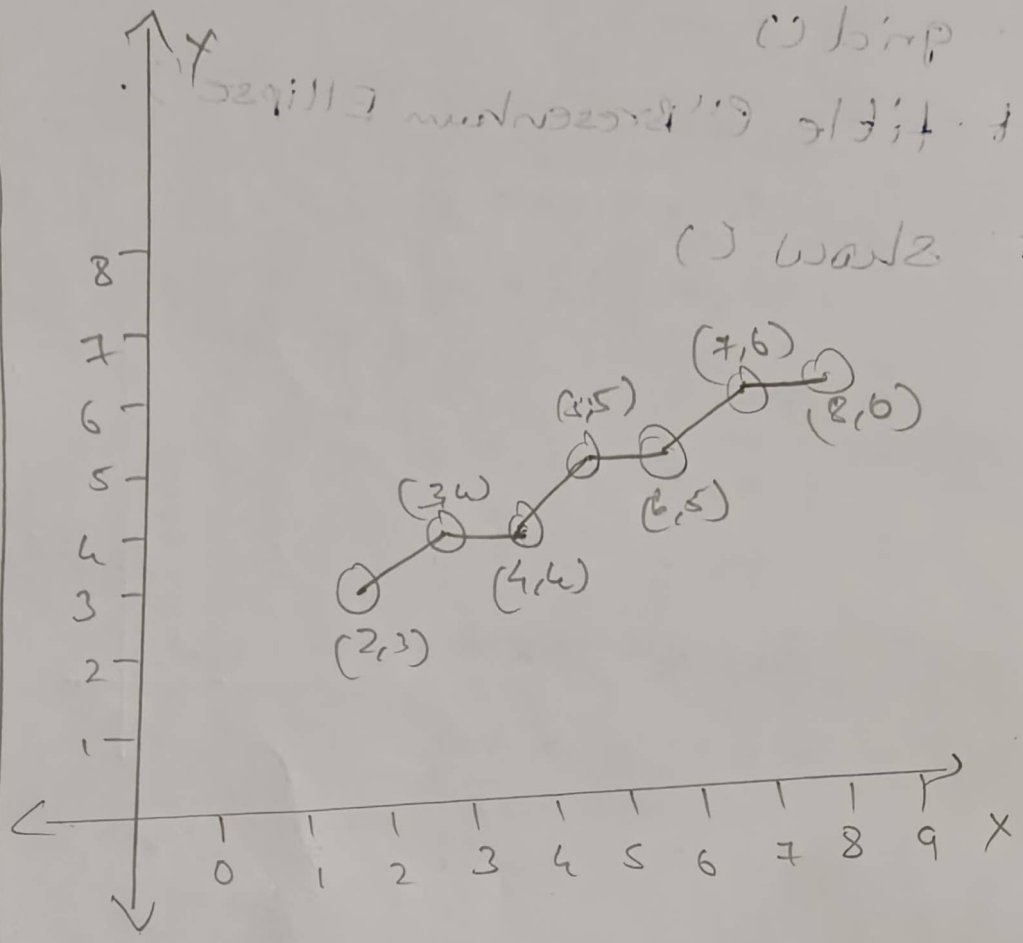
```
import matplotlib.pyplot as plt
DDA (x1, y1, x2, y2):
def
dx = x2 - x1
dy = y2 - y1
steps = max(abs(dx), abs(dy))
x_increment = dx/steps
y_increment = dy/steps
x = x1
y = y1
x_points = []
y_points = []
for i in range(steps+1):
    x_points.append(round(x))
    y_points.append(round(y))
    x += x_increment
    y += y_increment
```

```
plt.figure (figsize =(6,6))
plt.plot (x-points, y-points, color = 'blue',
          linewidth = 2, marker = 'o')
plt.title ("DDA line drawing algorithm")
plt.xlabel ("x-axis")
plt.ylabel ("y-axis")
plt.grid (True)
plt.gca().set_aspect ("equal",
 adjustable = "box")
plt.show()
x1 = int (input ("Enter x1: "))
y1 = int (input ("Enter y1: "))
x2 = int (input ("Enter x2: "))
y2 = int (input ("Enter y2: "))
```

Result:

Hence the implementation DDA line drawing Algorithm is verified successfully

OUTPUT





Expt 5 Draw a Rectangle using Bresenham Algorithm

Aim:

To implement drawing Rectangle using Bresenham Algorithm.

code:

```
import matplotlib.pyplot as plt
def Bresenham (x1, y1, x2, y2):
    dx, dy = abs(x2-x1), abs(y2-y1)
    sx, sy = (1 if x1<x2 else -1),
    (1 if y1 <y2 else -1)
    err = dx-dy
    p = []
while True:
    p.append ((x1, y1))
    if x1 == x2 and y1 == y2:
        break
    e = 2 * err
    if e > -dy:
        x1 += sx
    if e < dx:
        err += dx
        y1 += sy
```

```
return p

x, y, w, h = 2, 2, 8, 5

p = bresenham (x, y, x+w, h)
p += bresenham (x+w, y, x+w, y+h)
p += bresenham (x+w, y+h, x, y+h)
p += bresenham (x, y+h, x, y)

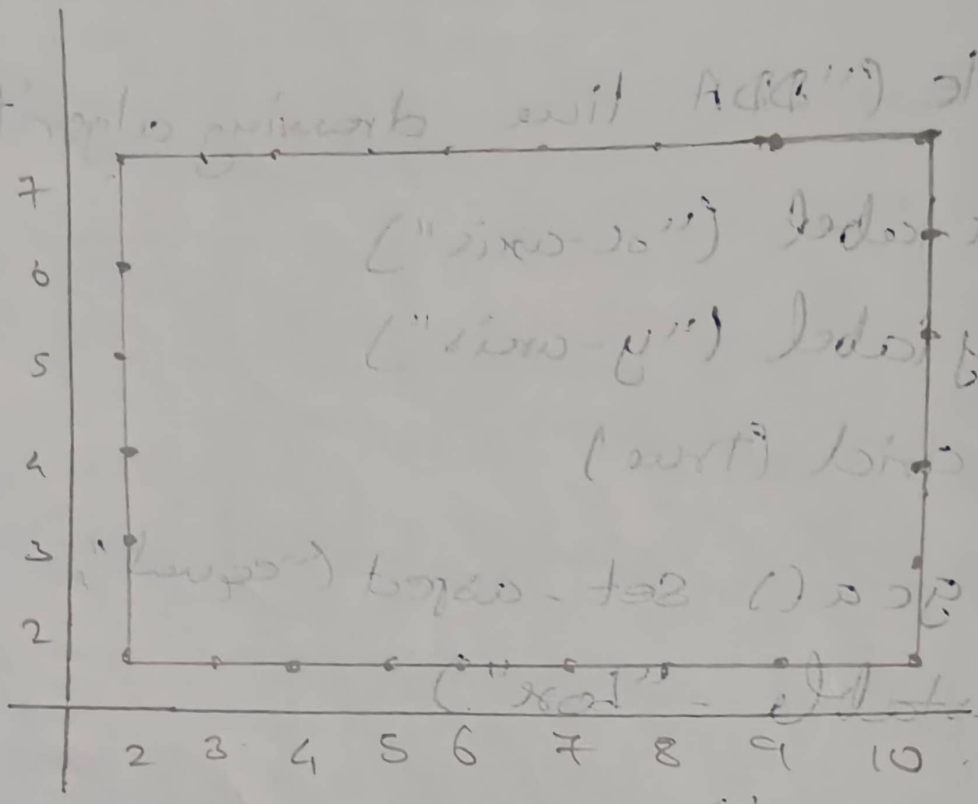
x, y = zip (*p)

plt.Fill (x, y, color = "skyblue")
plt.scatter (x, y, color = "blue")
plt.grid ()
plt.axis ("equal")
plt.show ()
```

Result:

Hence the drawing of rectangle is verified successfully

OUTPUT:



Exp 6 Draw a traffic light using circles

Aim:

To implement the drawing of traffic light using circles.

code:

```
import matplotlib.pyplot as plt
From matplotlib.patches import
Circle, Rectangle
fig, ax = plt.subplots ()
body = Rectangle ((0,0 1,4,10, facecolor
= "black", edge color = "gray")
ax.add-patch (body)
red = Circle ((2,7.5), 1.2, facecolor="red",
edgecolor ="white")
ax.add-patch(red)
yellow = Circle ((2,5), 1.2, facecolor
="yellow", edgecolor = "white")
ax.add-patch (yellow)
green = Circle((2,2.5), 1.2, facecolor
="green", edgecolor = "white")
```

```
ax.add_patch(green)

pole = Rectangle((1.7, -4), 0.6, 4,
facecolor="black")
ax.add_patch(pole)

ax.plot([-2, 6], [0, 0], color="gray",
linewidth=3)

ax.set_xlim(-2, 6)

ax.set_ylim(-4, 11)

ax.set_aspect("equal")

ax.axis("off")

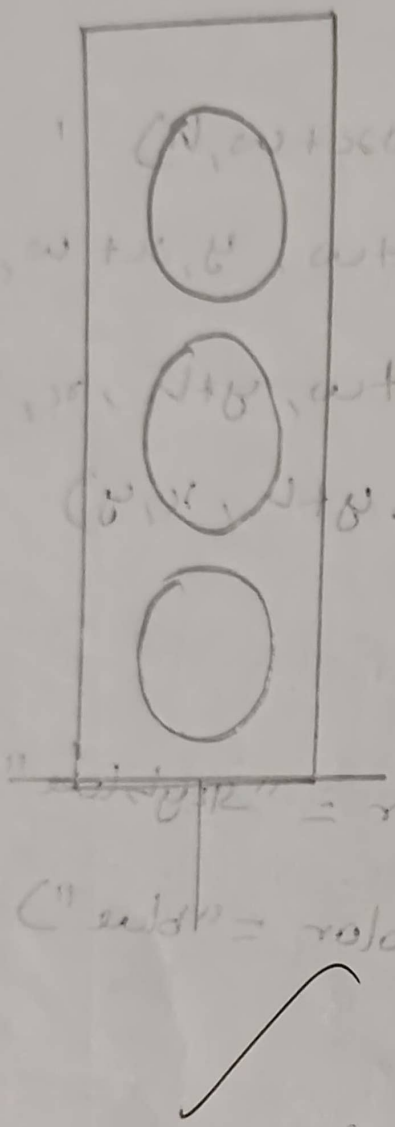
plt.title("Traffic signal")

plt.show()
```

Result:

Hence the drawing of traffic signals using circles is verified successfully

OUTPUT



Exp 7 Draw a smiley face using circles includes nose, mouth and eyes

Aim:

To implement the smiley face using circles

Code:

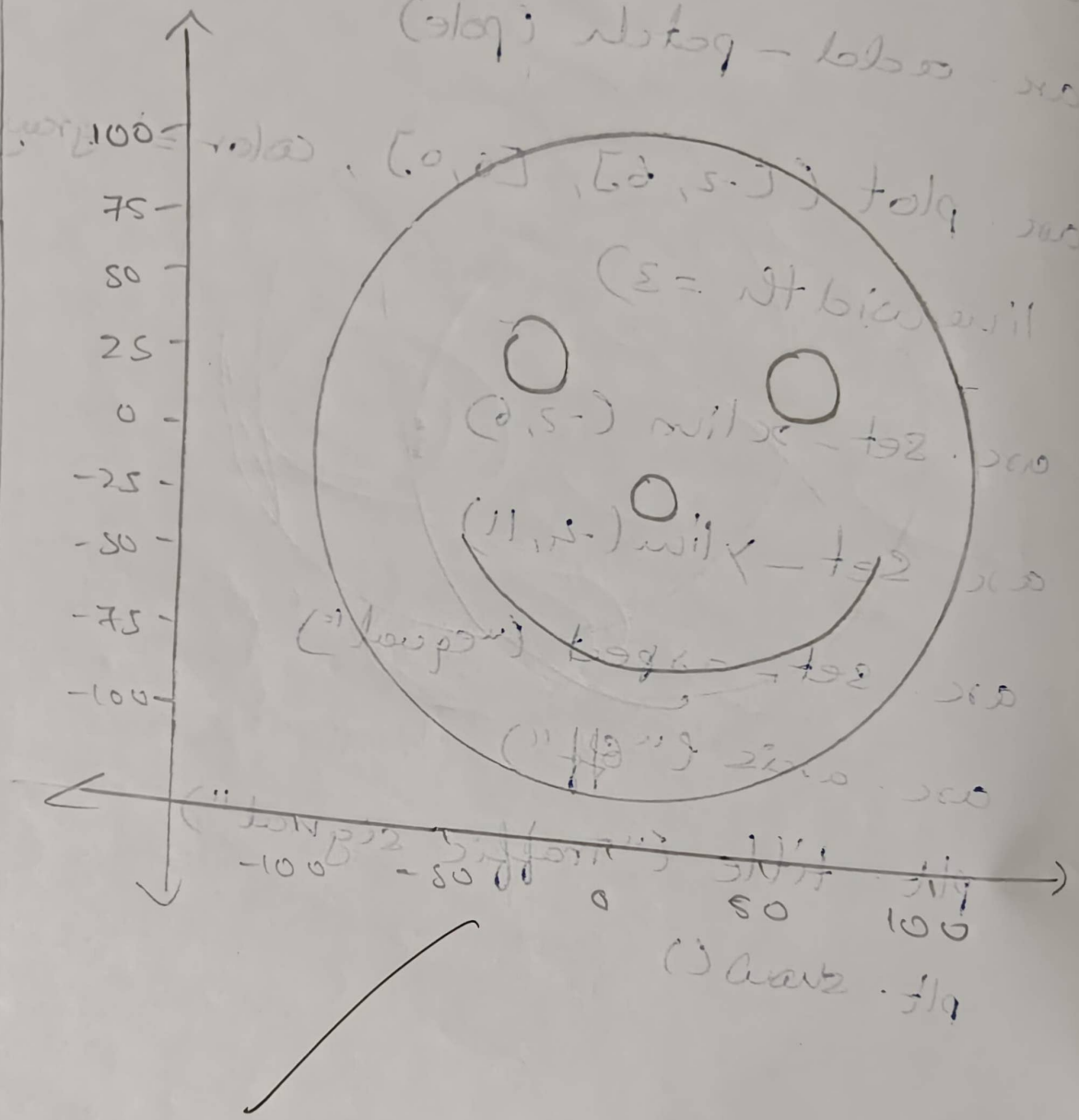
```
import matplotlib.pyplot as plt
def circle(xc, yc, r):
    x, y = 0, r
    p = 1 - r
    px, py = [], []
    while x <= y:
        pts = [(x, y), (-x, y), (x, -y), (-x, -y),
               (y, x), (-y, x), (y, -x), (-y, -x)]
        for dx, dy in pts:
            px.append(xc + dx)
            py.append(yc + dy)
        if p < 0:
            p += 2 * x + 3
        else:
            p += 2 * (x - y) + 5
            y -= 1
        x += 1
    return px, py
```

```
plt.figure()
x, y = circle(0,0,40)
plt.scatter(x,y,s=1)
x,y = circle(-15,15,5)
plt.scatter(x,y,s=1)
x,y = circle(15,15,5)
plt.scatter(x,y,s=1)
x,y = circle(0,0,3)
plt.scatter(x,y,s=1)
x,y = circle(0,-5,20)
for i in range(len(y)):
    if y[i] < -5:
        plt.scatter(x[i], y[i], s=1)
plt.axis("equal")
plt.show()
```

Result:

Hence the implementation of smiley face using the circles is implemented successfully

OUTPUT



Exp-8 Drawing of olympic logo using ellipse

Aim:

To implement drawing of olympic logo using ellipse

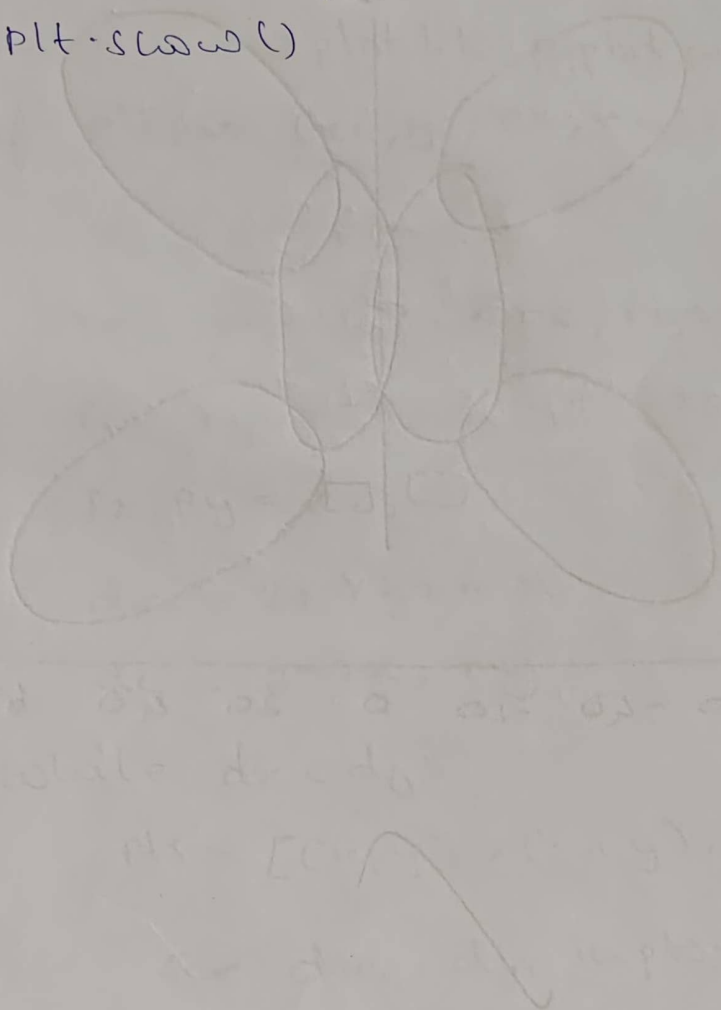
Code:

```
import matplotlib.pyplot as plt
def ellipse(xc, yc, rx, ry):
    x, y = 0, ry
    rx2, ry2 = rx*rx, ry*ry
    p1 = ry2 - rx2*ry + 0.25*rx2
    px, py = [], []
    dx = 2*ry2*x
    dy = 2*rx2*y
    while dx < dy:
        pts = [(x, y), (-x, y), (x, -y),
               (-x, -y)]
        for dx1, dy1 in pts:
            px.append(xc+dx1)
            py.append(yc+dy1)
        if p1 < 0:
            x += 1
            dx = 2*ry2*x
            p1 += dx + ry2
```

```
rings = [(-40,20), (0,20), (40,20), (-20,0),
         (20,0)]

for xc, yc in rings
    x, y = ellipse (xc, yc, 15, 20)
    plt.scatter (x, y, s=1)

plt.axis ("equal")
plt.show()
```



Result:

Hence the implementation of drawing olympic logo using ellipse is verified successfully

Exp-9 Drawing Butterfly using ellipse

Aim:

To implement the drawing of Butterfly using ellipse

code:

```
import matplotlib.pyplot as plt
def ellipse(xc, yc, rx, ry):
    x, y = 0, ry
    rx2, ry2 = rx*rx, ry*ry
    p1 = ry2 - rx2*ry + 0.25*rx2
    px, py = [], []
    dx = 2*ry2*x
    dy = 2*rx2*y
    while dx<dy:
        pts = [(x,y), (-x,y), (x,-y), (-x,-y)]
        for dx1, dy1 in pts:
            px.append(xc+dx1)
            py.append(yc+dy1)
        if p1<0:
            x+=1
            dx = 2*ry2*x
            p1+=dx+ry2
```

else:

```
x+ = 1
y- = 1
dxc = 2 * ry2 * x
dy = 2 * rx2 * y
p1 += dx - dy + ry2

p2 = (ry2 * (x + 0.5) ** 2) + (ry2 * (y - 1) ** 2) -
(rx2 * ry2)
while y >= 0:
    pts = [(x, y), (-x, y), (x, -y), (-x, -y)]
    for dx1, dy1 in pts:
        px.append(xc + dx1)
        py.append(yc + dy1)
    if p2 > 0:
        y- = 1
        dy = 2 * rx2 * y
        p2 += rx2 - dy
    else:
        y- = 1
        x+ = 1
        dxc = 2 * ry2 * x
        dy = 2 * rx2 * y
        p2 += dx - dy + rx2
return px, py
plt.figure()
```

```
parts = [(-20, 20, 20, 10),
         (20, 20, 20, 10),
         (-20, -10, 15, 10),
         (20, -10, 15, 10)]

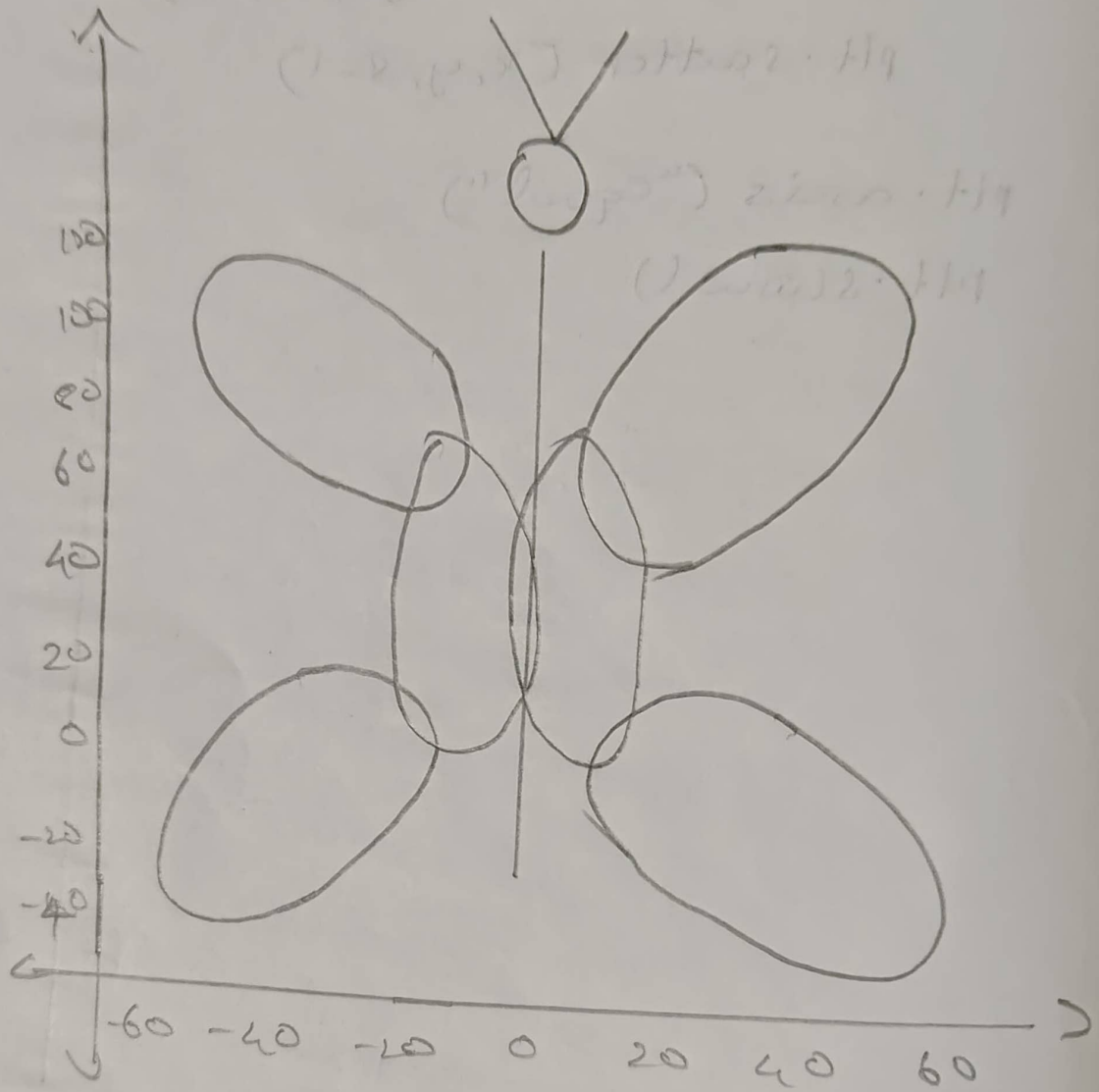
for xc, yc, rx, ry in parts:
    x, y = ellipse(xc, yc, rx, ry)
    plt.scatter(x, y, s=1)

plt.plot([0,0], [-25,35])
plt.plot([0,-10], [35,45])
plt.plot([0,10], [35,45])
plt.axis("equal")
plt.show()
```

Result:

Hence the implementation of drawing the butterfly using ellipse is verified successfully.

OUTPUT:



Exp-10 Flood fill algorithm

Aim:

To implement flood fill algorithm using scanline function

code:

```
import cv2
import numpy as np
img = np.zeros((400,400,3),np.uint8)
cv2.rectangle(img,(100,100),(300,300),
  (255,255,255),2)
def fill(x,y):
stack =[(x,y)]
while stack:
    x,y = stack.pop()
    l=x
    while l>0 and all(img[y,l]==0):l-=1
    r=x
    while r<399 and all(img[y,r]==0):r+=1
    img[y,l:r]=[0,255,0]
    for i in range(l,r):
     if y>100 and all(img[y-1,i]==0):
     stack.append((i,y-1))
```

Fill (200, 200)

cv2.imshow ("scanline Flood Fill", img)

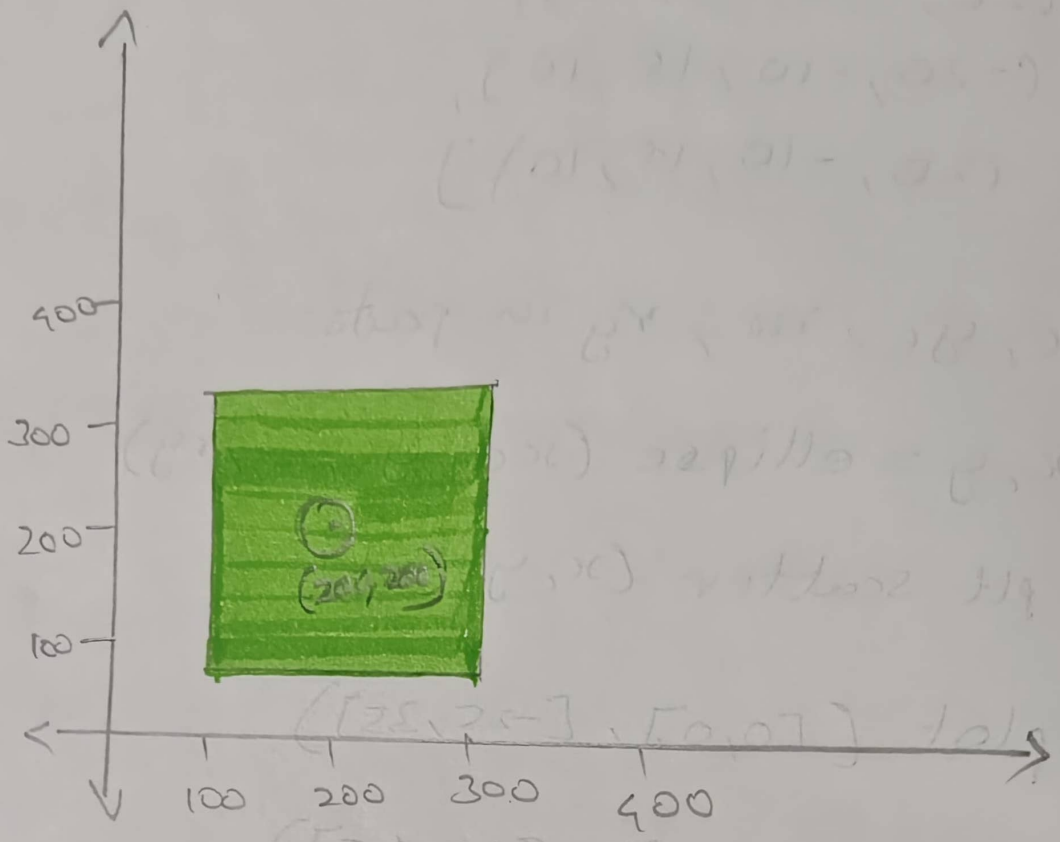
cv2.waitKey (0)

cv2.destroyAll windows ()

Result:

Hence the Flood Fill algorithm is verified successfully

OUTPUT:



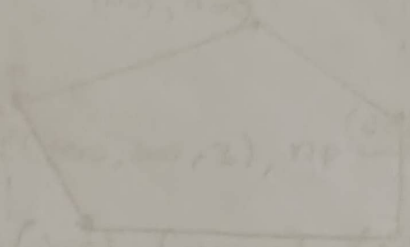
Exp-11 2D Transformations

Aim:

To implement 2D transformations Algorithm using Python

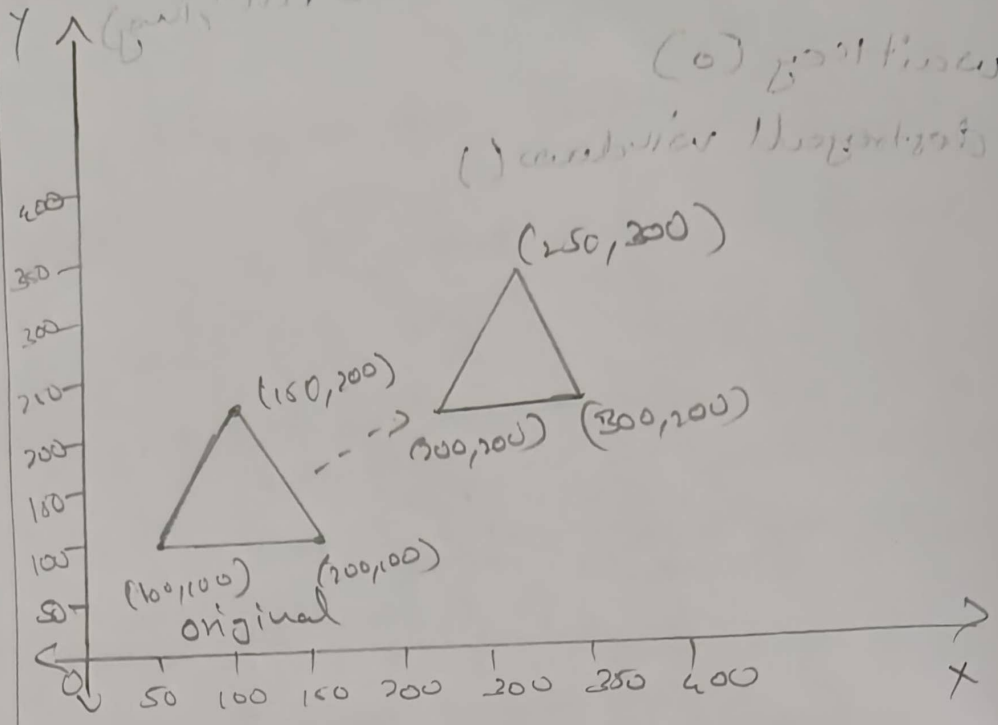
code:

```
import cv2
import numpy as np
img = np.zeros((400,400,3),np.uint8)
p = np.array([[100,100],[180,50],[200,100]],
np.int32)
p1 = p+[100,100]
p2 = (p*1.5).astype(int)
M = cv2.getRotationMatrix2D((150,100),45,1)
p3 = cv2.transform(np.array([p]),M)
[0].astype(int).
cv2.polylines(img,[p],True,(255,255,255),2)
cv2.polylines(img,[p1],True,(0,255,0),2)
cv2.polylines(img,[p2],True,(0,0,255),2)
cv2.polylines(img,[p3],True,(0,0,255),2)
cv2.imshow("2D transformations",img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

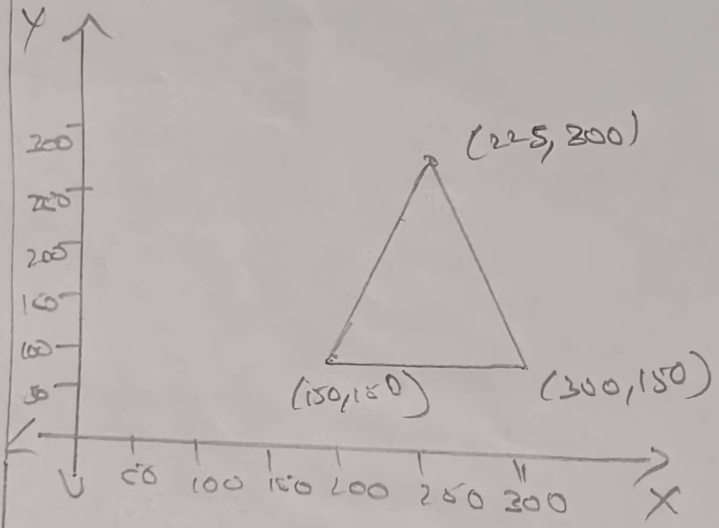


result: Hence the 2D transformation Algorithm is verified successfully

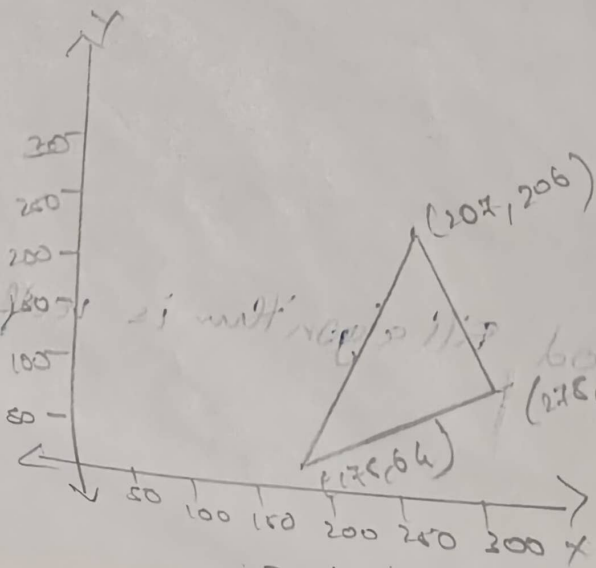
OUTPUT:



Translation



Scaled



Exp-12 polygon clipping

Aim:

To import polygon clipping using python

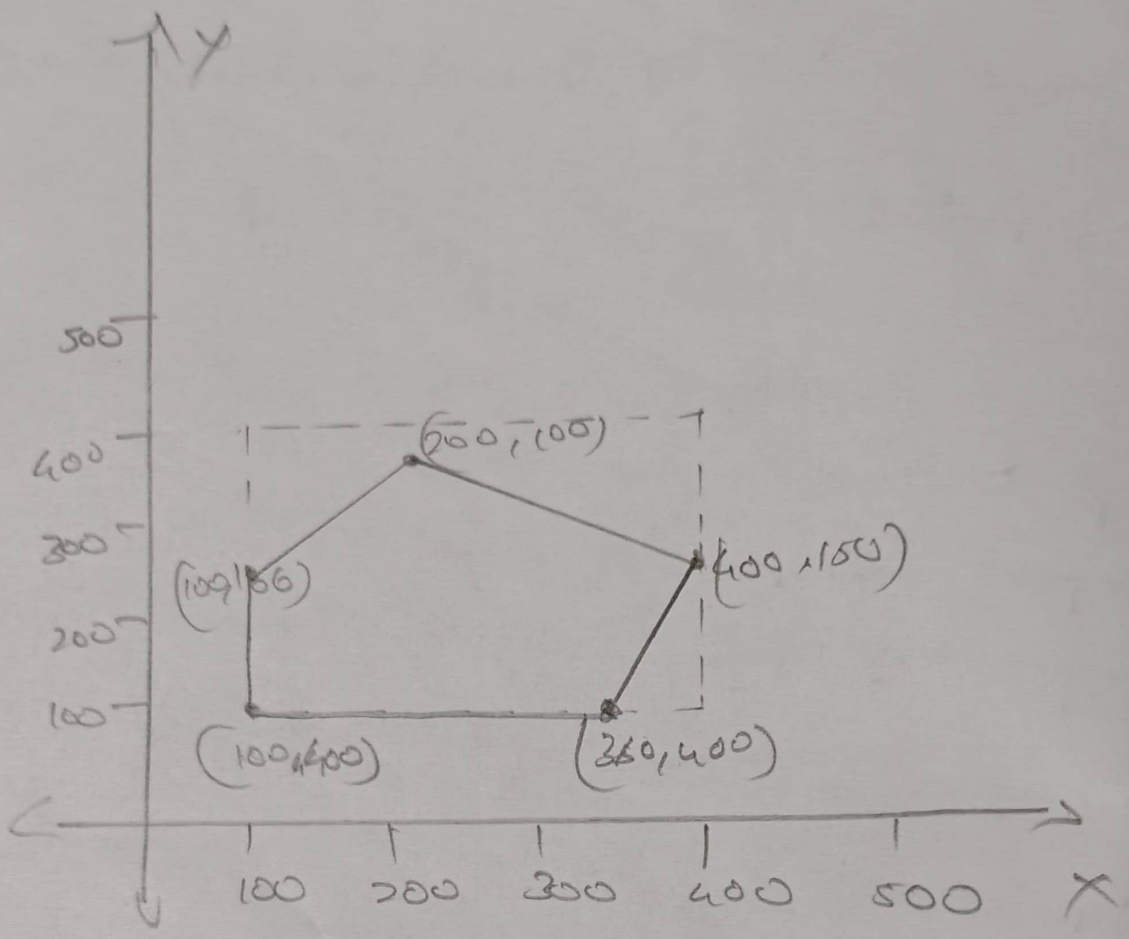
Code:

```
import cv2
import numpy as np
img = np.zeros((500,500,3), np.uint8)
cv2.rectangle(img, (100,100), (400,400),
(255,255,255), 2)
p = np.array([[50,100],[200,50],[450,150],[350,400],
[100,400]], np.int32)
q = cv2.clipLine([(100,100,300,300),(0,0),
(0,0))][0] if false else p
mask = np.zeros((500,500),np.uint8)
cv2.fillpoly(mask,[p],255)
window = np.zeros((500,500),np.uint8)
cv2.rectangle(window,(100,100),(400,400),
255,-1)
result = cv2.bitwise_and(mask,window)
img[result>0] = (0,255,0)
cv2.imshow("polygon clipping",img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Result:

Hence the implementation of Polygon clipping is verified successfully

OUTPUT:


Y
500
400
300
200
100
(300,100)
(100,166)
(400,150)
(100,400)
(350,400)
100
200
300
400
500
X

Exp no: 13 - Basic 3D transformation

Aim:

To implement Basic 3D transformations in python

Code:

```
import matplotlib.pyplot as py
from mpl_toolkits.mplot3d import Axes3D
import numpy as np

p = np.array([[0,0,0],[1,0,0],[1,1,0],
    [0,1,0],[0,0,1],[1,0,1],[1,1,1],
     [0,1,1]])

TX,TY,TZ = 2,1,1
translated = p+[TX,TY,TZ]

Sx,Sy,Sz = 2,2,2
Scaled = p*[Sx,Sy,Sz]

angle = np.radians(45)
R = np.array([[np.cos(angle), -np.sin(
                                angle),
                                0],
    [np.sin(angle), np.cos(angle), 0],
     [0,0,1]])

rotated = P@R.T
```

```
edges = [(0,1), (1,2), (2,3), (3,0),
         (4,5), (5,6), (6,7), (7,4),
         (0,4), (1,5), (2,6), (3,7)]

def draw(ax, points, title):
    for a, b in edges:
        ax.plot([points[a,0],
                 points[b,0]],
                [points[a,1], points[b,1]],
                [points[a,2], points[b,2]])

    ax.set_title(title)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_zlabel("z")

fig = plt.figure(figsize=(15,4))

ax1 = fig.add_subplot(131, projection='3d')
draw(ax1, P, "Original Cube")

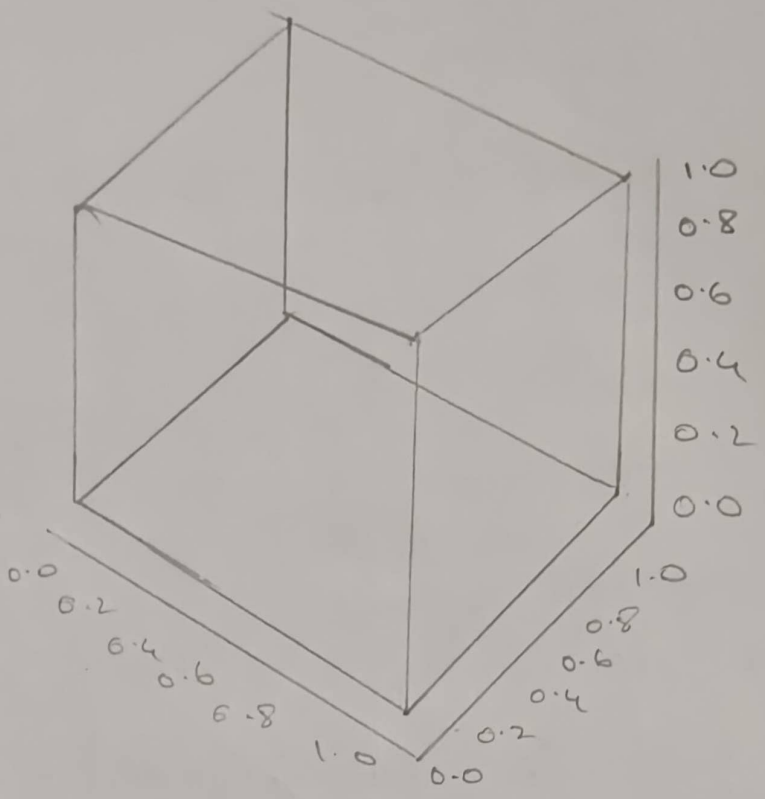
ax2 = fig.add_subplot(132, projection='3d')
draw(ax2, translated, "Translation")

ax3 = fig.add_subplot(133, projection
    ='3d')
draw(ax3, rotated, "Rotation")
```

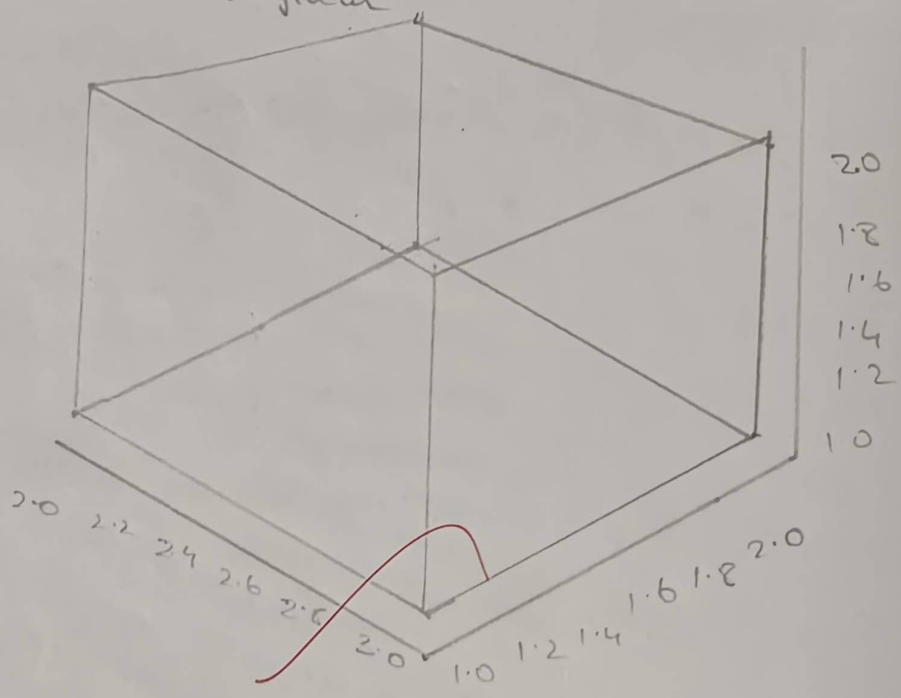
Result:

Hence to implement the basic 3d transformation in python is verified successfully.

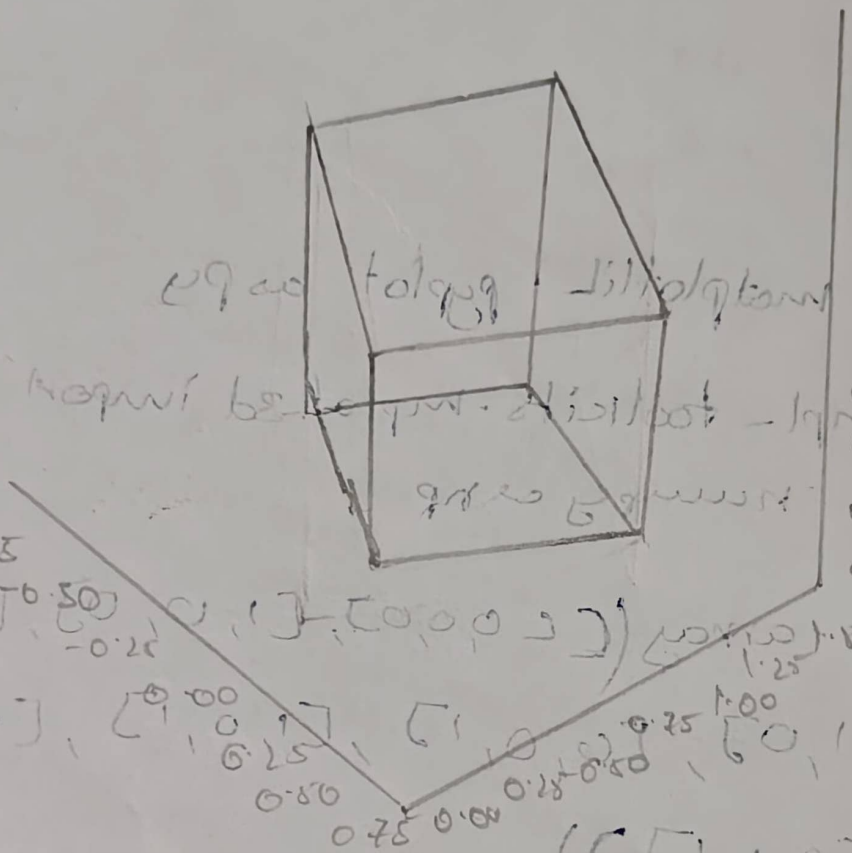
OUTPUT



original



Translation



Rotated

Experiment: Create and Render 3D objects using polygons and Apply basic color and shading Techniques

Aim:

To create and Render 3D objects (such as cube or sphere) using polygonal modelling and apply

Procedure:

1. Define the 3D object using polygon faces
2. Assign color values to faces or vertices
3. Apply simple shading models
   - flat shading
   - vertex coloring
4. Use 3D plotting to render the object with shading
5. Visualize the effect of light using normal approximation

Program:

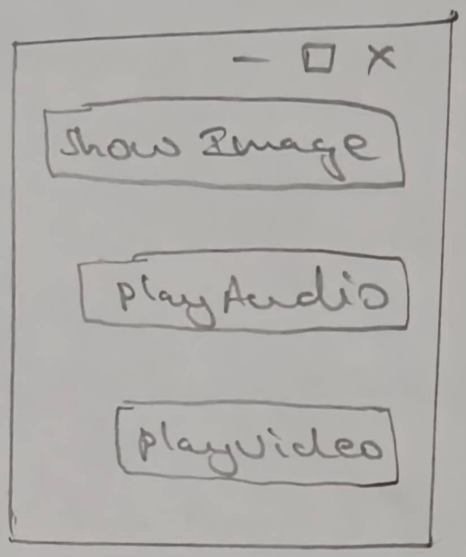
```
import matplotlib.pyplot as py
from mpl_toolkits.mplot3d.art3d
import poly3dcollection
import numpy as np
vertices = np.array([[0,0,0],[1,0,0],[1,1,0]
  [0,1,0],[0,0,1],[1,0,1],[1,1,1],[0,1,1]
faces = [[vertices[j] for j in [0,1,2,3]],
```

```
[vertices[j] for j in [4,5,6,7]],
[vertices[j] for j in [0,1,5,4]],
[vertices[j] for j in [2,3,7,6]],
[vertices[j] for j in [1,2,6,5]],
[vertices[j] for j in [4,7,3,0]]]
```

Result:

3D polygonal objects were successfully created and rendered. Basic color and flat shading techniques were applied to enhance visual representation

OUTPUT



Exp: #16 Create a multimedia Application that integrates images, sound and video in simple User Interface

Aim:

To develop a simple multimedia Application that integrates image sound, video

Program:

```
import tkinter as tk
from tkinter import filedialog
from PIL import image, ImageTk
from playsound import playsound
import cv2
import threading
def play_audio():
playsound('sample_audio.mp3')
def play_video():
  cap = cv2.VideoCapture('sample_video')
  while cap.isOpened():
      ret, frame = cap.read()
      if not ret:
          break
      cv2.imshow("video", frame)
      if cv2.waitKey(25) & 0XFF ==
      ord('q'):
          break
```

```
cap.release()
cv2.destroyAllWindows()

def load_image():
    img = Image.open("sample_image.jpg")
    img = img.resize((300, 300))
    img_tk = ImageTk.PhotoImage(img)
    panel.configure(image=img_tk)
    panel.image = img_tk

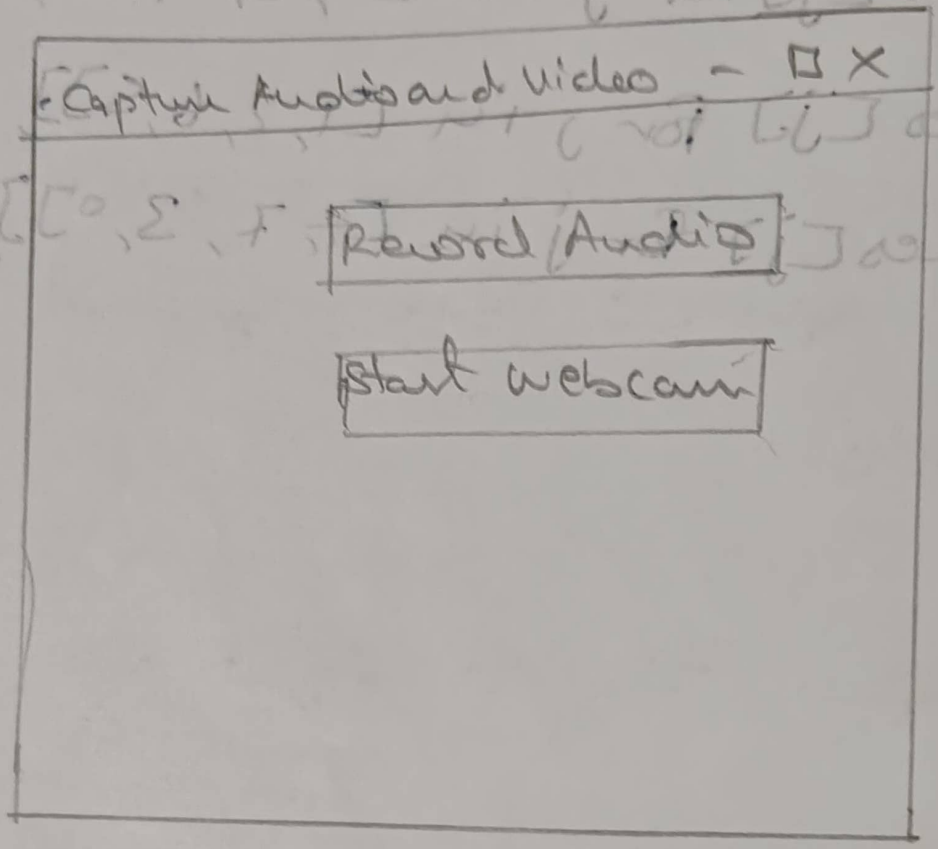
window = tk.Tk()
window.title("Multimedia App")
btn_img = tk.Button(window, text=
"Show Image", command=load_image)
btn_audio = tk.Button(window,
    text="Play Audio", command=
        lambda:
threading.Thread(target=play_audio)
start()
btn_video = tk.Button(window, text
="Play Video", command=lambda:
threading.Thread(target=play_
    video).start())
```

```
Panel = tk.Label(window)
Panel.pack()

btn_img.pack(pady=5)
btn_audio.pack(pady=5)
btn_video.pack(pady=5)
window.mainloop()
```

Result: A multimedia Application was successfully created that integrates image, Audio and video functionality into single interactive interface

OUTPUT



Exp 17 capture video/Audio from webcam or Microphone and display on Multimedia Interface

Aim:

To develop a program that captures

- Live video from the webcam
- Live audio from the microphone

Program:

```
import cv2
import sounddevices as sd
from scipy.io.wavfile import write
import threading
import tkinter as tk

def record_audio():
    duration = 5
    fs = 44100
    print("Recording audio")
    audio = sd.rec(int(duration * fs),
    sd.wait()
    write('recorded_audio.wave')
    print("Audio recording saved.")

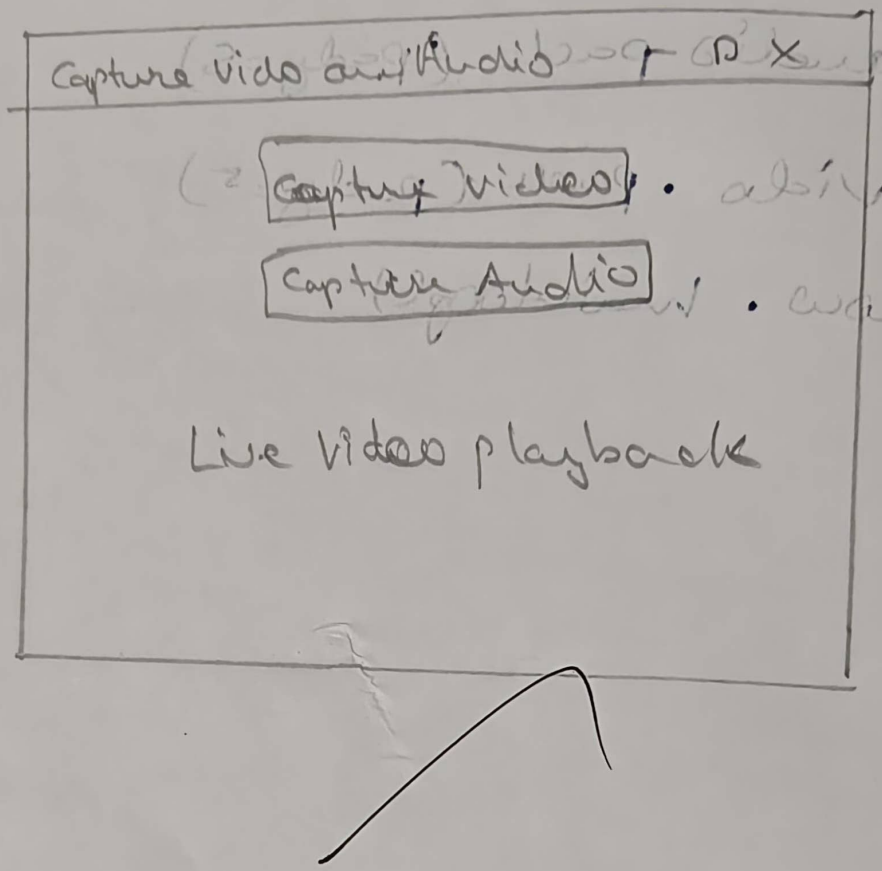
def capture_video():
    cap = cv2.VideoCapture(0)
    print("Press 'q' to stop video.")
    while True:
        ret, frame = cap.read()
```

```
if not ret:
    break
cv2.imshow('Webcam Feed', Frame)
if-cv2.waitKey(1) &0xFF == ord('q'):
    break
cap.release()
cv2.destroyAllWindows()
window = tk.Tk()
window.title("Capture Audio.and Video")
btn-audio = tk.button(window, text
='Record-audio')
btn_video = tk.button(window, text
='start Webcam'),
threading.thread(target = Capture_Video)
.start()
btn.audio.pack(pady=10)
btn.avido.pack(pady=10)
window.mainloop
```

Result:

Live video and Audio were successfully captured using webcam and microphone

output


Capture Video and Audio
Capture Video
Capture Audio
Live video playback

Exp 12 Create an Application to send and Recieve Multimedic Messages.

Aim:

To develop a basic chat-like application that allows sending image, video, audio, text

program:

```
import tkinter as tk
from tkinter import filedialog, scrolledtext
from PIL import image, imageTk
from playsound import playsound
import threading
def send_text():
    msg = text_entry.get()
    if msg:
        chat_box.insert(tk.END, f"You:
            {msg}\n")
        text_entry.delete(0,tk.end)'

def send_image():
    path= filedialog.askopenfilename
(filetypes=[("Image files", "*.jpg.*.png.
*.jpeg")])
```

```
if path:
    chat_box.insert(tk.END, f"You sent an image: {path}\n")
    img = Image.open(path)
    img.thumbnail((100,100))
    img_tk = ImageTk.PhotoImage(img)
    image_label = tk.Label(chat_frame, image=img_tk)
    image_label.image = img_tk
    image_label.image = img_tk
    image_label.pack()
    chat_box.yview(tk.END)
else:
    chat_box.insert(tk.END, f"You sent an audio message: {path}\n")

window = tk.Tk()
window.title("Messaging App")
window.geometry("400x500")

chat_frame = tk.Frame(window)

chat_box = scrolledtext.ScrolledText(chat_frame, wrap=tk.WORD, width=50, height=20)

chat_box.pack()
chat_frame.pack(pady=10)
```

```
btn.text.pack(side=tk.LEFT)
btn.text.pack(side=tk.LEFT)
btn.Audio.pack(side=tk.LEFT)

window.mainloop()
```

Result:

Hence the Multimedia Messaging App is built and verified successfully

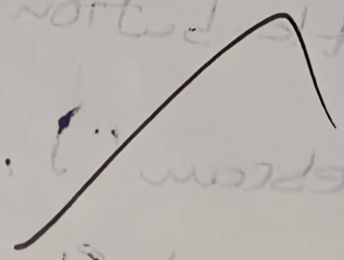
Messaging APP - ☐ X

You: Hello

you: Sent an image
C:/user/downloads/img

you: Sent an audio

C:/user/downloads/audio



Exp 19 Create a 3D Model using Blender and Apply Basic Shading Texture

Aim:

To create a simple 3D model using Blender

Procedure:

1. Launch Blender
2. Model creation
   * use Add > Mesh to insert basic shapes like cube, cylinder, plane etc
   * combine these objects to form a house or a similar structure.
   * use Edit mode for extruding and shaping
3. Shading
   - switch to Shading
   - Add a new material for each object
   - Adjust base color, Specular, Roughness

4. Texture and Mapping:

- In the Shader Editor, add an Image texture node.
- Load an external image
- connect it to the base color of the principled BSDF Shader
- use UV editing to align the texture

5. Lighting and camera:

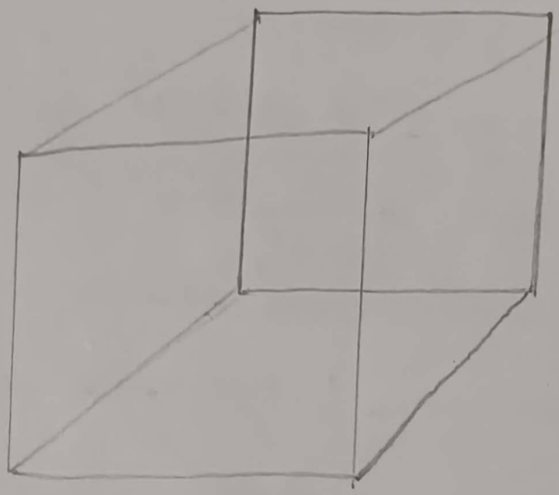
- Add Light sources (sun or point Light).
- position the camera to frame the model

6. Render settings:

- choose cycles or Eevee rendering engine
- Adjust resolution and sampling
- Click Render → Render Image to output

Result:

A 3D object was modelled and verified successfully in Blender



Completed

12/04/26